



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y. 2026-27 Semester-3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



Practical-5

Practical AIM:

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

5.1

Problem Defination:

A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation. Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

Output:

```
PS D:\DSA\Practical5\output> & .\'5.1.exe'  
Playlist: Song C Song A Song D Song B  
Total songs: 4  
Playlist: Song A Song D Song B
```

Difficulty Faced:

Maintaining both `prev` and `next` pointers correctly during insertion and deletion, especially when the list is empty or contains only one song.

Solution:

A doubly linked list is used to store songs. Each node contains the song name, a `prev` pointer, and a `next` pointer. The required operations—add at beginning, add at end, insert after a song, remove first, count, and display—are implemented by updating these links. If the song is not found, no insertion is performed.

Outcome:

The doubly linked playlist was successfully implemented with insertion at the beginning, insertion at the end, insertion after a specific song, deletion of the first song, counting, and display operations. The program correctly maintains both forward and backward links and handles special cases such as an empty playlist and a song that is not found.

5.2

Problem Definition:

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation. Describe how

your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

Output:

```
PS D:\DSA\Practical5\output> & .\5.2.  
Choose Circular Linked List:  
1. Singly Circular Linked List  
2. Doubly Circular Linked List  
Enter choice: 1  
  
Enter number of operations: 2  
  
1. Join student  
2. Leave student  
3. Display circle  
Enter operation: 1  
Enter student number: 2  
Enter position: 2  
2
```

Difficulty Faced:

The main difficulty was maintaining the circular connection after a student leaves, especially when removing the only student or the first student.

Solution:

Both singly circular and doubly circular linked lists are used to represent the students in a circle. Students can be inserted at any position and removed by updating the required links. After every operation, the list is displayed.

Outcome:

Successfully implemented join, leave, and display operations using both circular linked lists while keeping the circle connected after every operation.

Key Questions:

1. A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

Sol: In a doubly linked list, after deleting the front node, the new first node's prev **link must be set to null**. If we forget this, it may still point to the deleted node, causing an invalid or dangling link.

2. If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

Sol: Traversal should stop when we reach the starting node again. This proves that we have completed one full circle. If the last node points to `null`, we cannot return to the starting node, so the circular traversal condition will not work correctly.

Github Link:

<https://github.com/brindajivani/DSA/tree/main/Practical5>

